

Sistemas Operativos

Práctica 2

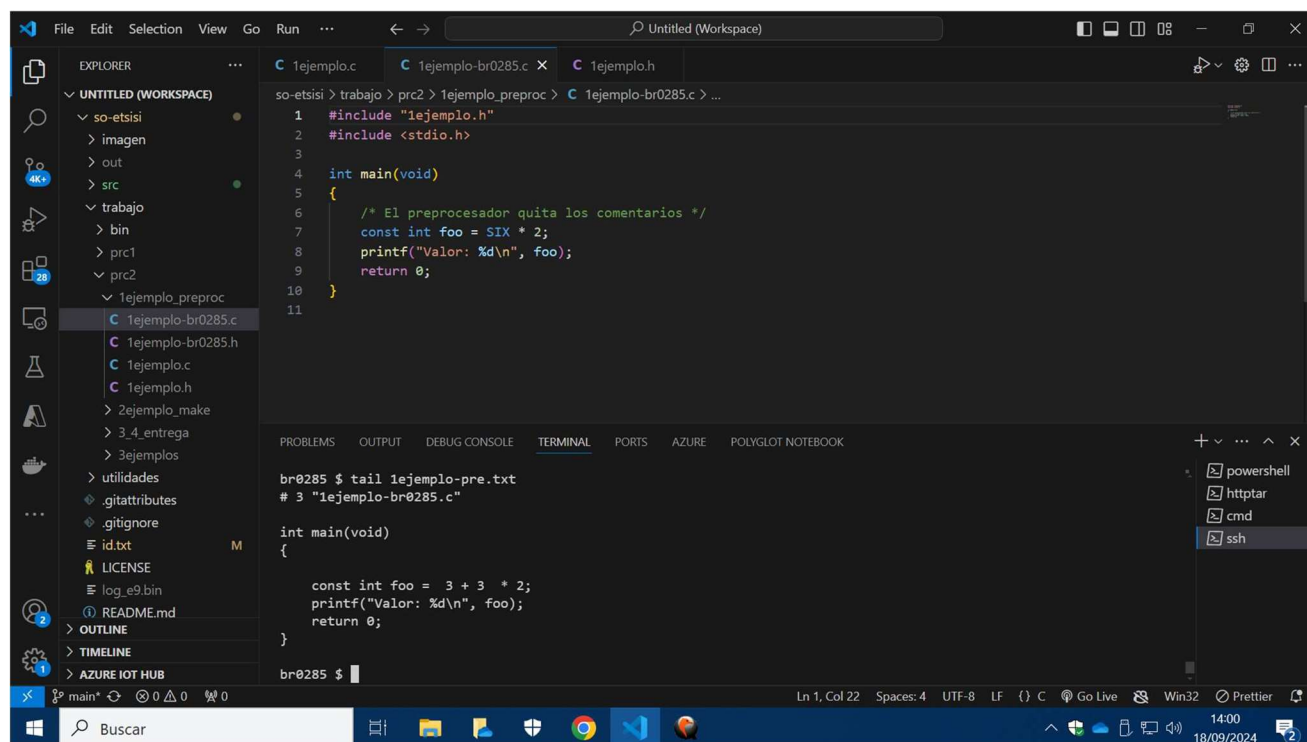
Fecha: 18/09

Alumno: Yzan Martinez Manzano

Correo: yzan.martinez.manzano@alumnos.upm.es

Introducción a C

A entregar – 1. Preprocesador Realiza una captura de pantalla en la que aparezca la ejecución de los comandos anteriores y se vea el contenido de 1ejemplo-_id_-pre.txt (usa el comando tail para mostrar solo las 10 últimas líneas) en la zona en la que está el contenido de printf. Acuérdate de que la captura, como todas las demás, aunque no se diga explícitamente, debe contener el escritorio completo, incluyendo la hora del sistema, el nombre del fichero Word en la barra de tareas, tu número de matrícula en el título de la ventana de qemu y el resultado de la ejecución de los anteriores comandos en la conexión ssh.



The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays a project structure with folders like 'so-etsi', 'imagen', 'out', 'src', 'trabajo', 'bin', 'prc1', 'prc2', and '1ejemplo_preproc'. The main editor window shows a C file named '1ejemplo-br0285.c' with the following code:

```
1 #include "1ejemplo.h"
2 #include <stdio.h>
3
4 int main(void)
5 {
6     /* El preprocesador quita los comentarios */
7     const int foo = SIX * 2;
8     printf("Valor: %d\n", foo);
9     return 0;
10 }
11
```

The TERMINAL panel at the bottom shows the command prompt for 'br0285' with the command 'tail 1ejemplo-pre.txt' and its output:

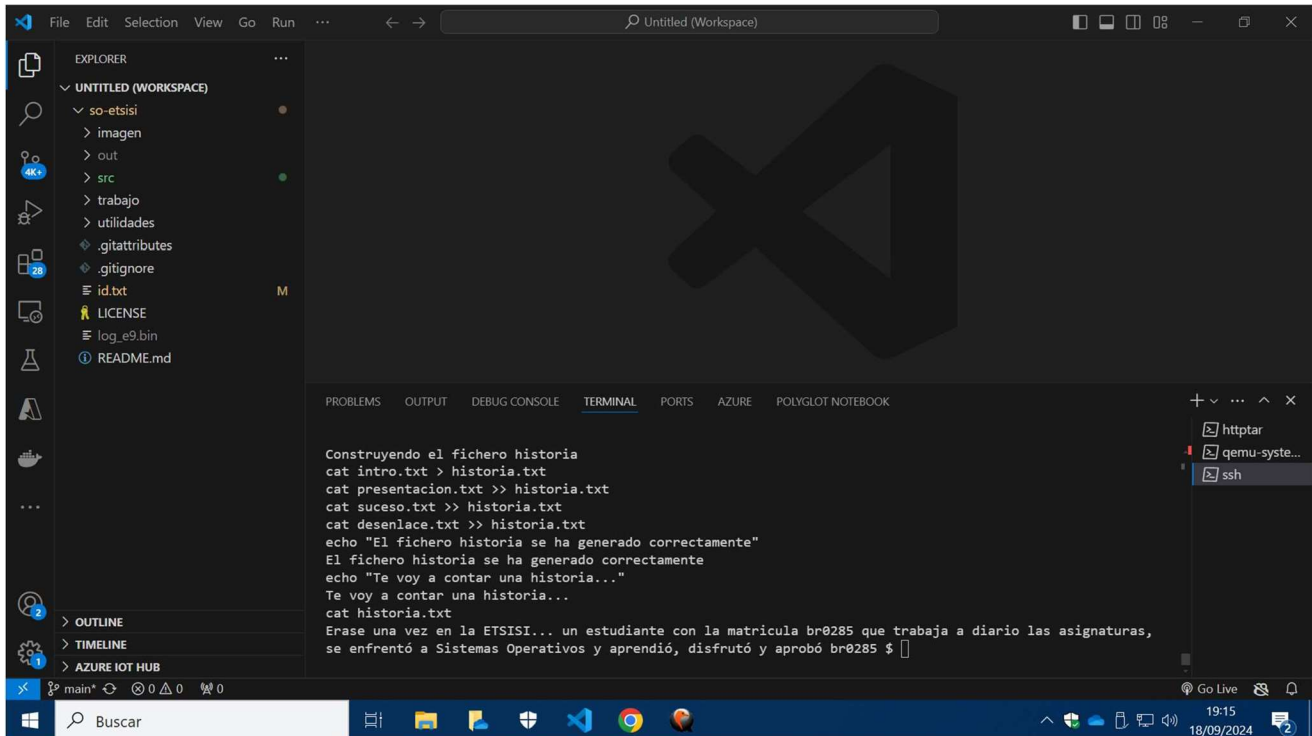
```
br0285 $ tail 1ejemplo-pre.txt
# 3 "1ejemplo-br0285.c"

int main(void)
{

    const int foo = 3 + 3 * 2;
    printf("Valor: %d\n", foo);
    return 0;
}
```

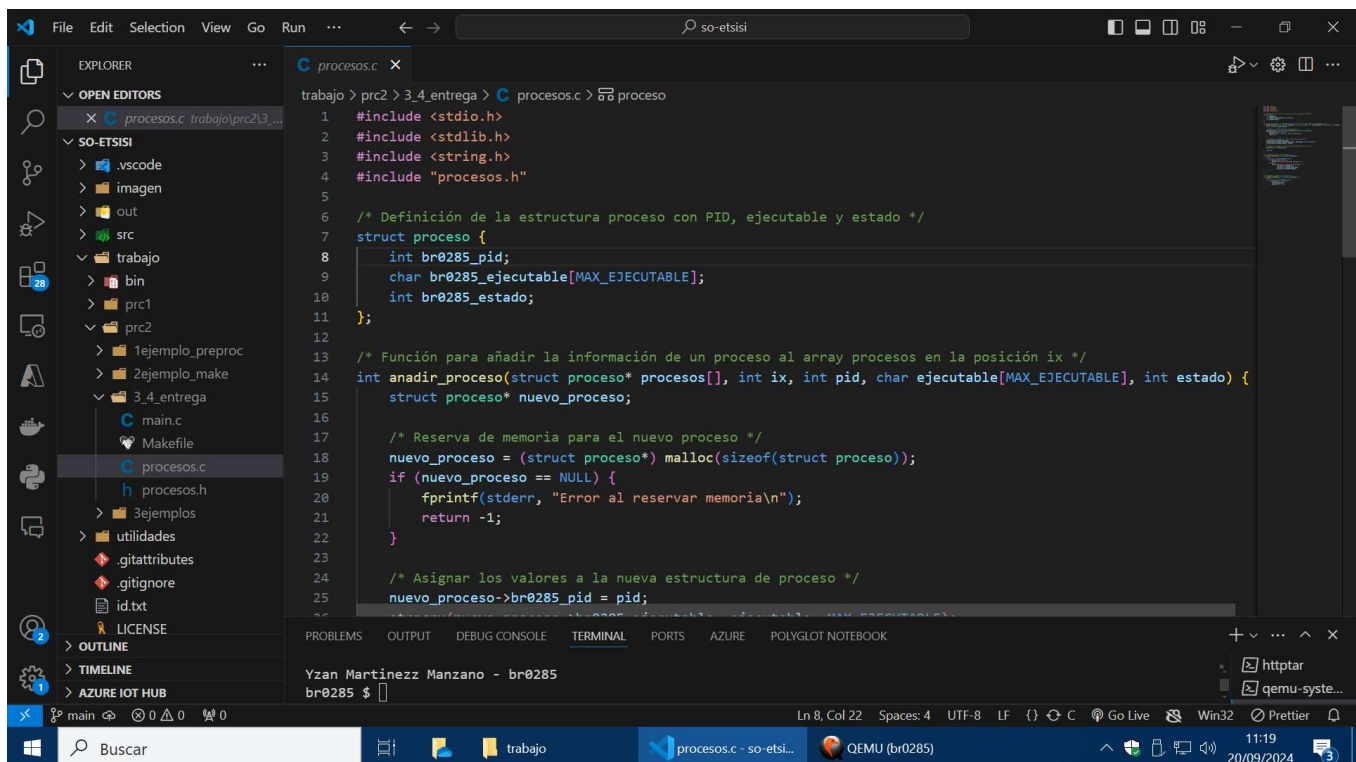
The status bar at the bottom indicates the current line and column (Ln 1, Col 22), encoding (UTF-8), and other settings. The Windows taskbar at the very bottom shows the search bar and various application icons.

A entregar – 2. make y Makefile Realiza una captura de pantalla (como siempre, de todo el escritorio) en la que aparezca la ejecución del último make y en la que se pueda apreciar las cadenas de texto modificadas y la construcción del fichero historia.



```
Construyendo el fichero historia
cat intro.txt > historia.txt
cat presentacion.txt >> historia.txt
cat suceso.txt >> historia.txt
cat desenlace.txt >> historia.txt
echo "El fichero historia se ha generado correctamente"
El fichero historia se ha generado correctamente
echo "Te voy a contar una historia..."
Te voy a contar una historia...
cat historia.txt
Erase una vez en la ETSISI... un estudiante con la matricula br0285 que trabaja a diario las asignaturas,
se enfrentó a Sistemas Operativos y aprendió, disfrutó y aprobó br0285 $
```

A entregar – 3. Estructura C para gestión proceso Realiza una captura de pantalla en la que deberá aparecer el código que has escrito en la declaración de la estructura.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "procesos.h"
5
6 /* Definición de la estructura proceso con PID, ejecutable y estado */
7 struct proceso {
8     int br0285_pid;
9     char br0285_ejecutable[MAX_EJECUTABLE];
10    int br0285_estado;
11 };
12
13 /* Función para añadir la información de un proceso al array procesos en la posición ix */
14 int anadir_proceso(struct proceso* procesos[], int ix, int pid, char ejecutable[MAX_EJECUTABLE], int estado) {
15     struct proceso* nuevo_proceso;
16
17     /* Reserva de memoria para el nuevo proceso */
18     nuevo_proceso = (struct proceso*) malloc(sizeof(struct proceso));
19     if (nuevo_proceso == NULL) {
20         fprintf(stderr, "Error al reservar memoria\n");
21         return -1;
22     }
23
24     /* Asignar los valores a la nueva estructura de proceso */
25     nuevo_proceso->br0285_pid = pid;
```

Visual Studio Code interface showing a C program named `procesos.c` being edited and run in a terminal.

File Explorer (Left):

- WORKSPACE: `trabajo`
 - `src`
 - `main.c`
 - `Makefile`
 - `bin`
 - `prc1`
 - `prc2`
 - `1ejemplo_preproc`
 - `2ejemplo_make`
 - `3.4_entrega`
 - `main.c`
 - `Makefile`
 - `procesos.c` (selected)
 - `procesos.h`
 - `3ejemplos`
 - `utilidades`
 - `.gitattributes`
 - `.gitignore`
 - `id.txt`

Code Editor (Center):

```
trabajo > prc2 > 3.4_entrega > C:procesos.c > 50 proceso
14 int anadir_proceso(struct proceso* procesos[], int ix, int pid, char ejecutable[MAX_EJECUTABLE], int estado) {
26     strncpy(nuevo_proceso->br0285_ejecutable, ejecutable, MAX_EJECUTABLE);
27     nuevo_proceso->br0285_estado = estado;
28
29     /* Colocar el proceso en la posición correspondiente del array */
30     procesos[ix] = nuevo_proceso;
31
32     return 0;
33 }
34
35 /* Función para listar los procesos */
36 void listar_procesos(struct proceso* procesos[]) {
37     int i;
38     for (i = 0; i < MAX_PROCESOS; i++) {
39         if (procesos[i] == NULL) {
40             printf("Posición de tabla %d: Vacía\n", i);
41         } else {
42             printf("Posición de tabla %d: %d %s %d\n", i,
43                 procesos[i]->br0285_pid,
44                 procesos[i]->br0285_ejecutable,
45                 procesos[i]->br0285_estado);
46         }
47     }
48 }
49
```

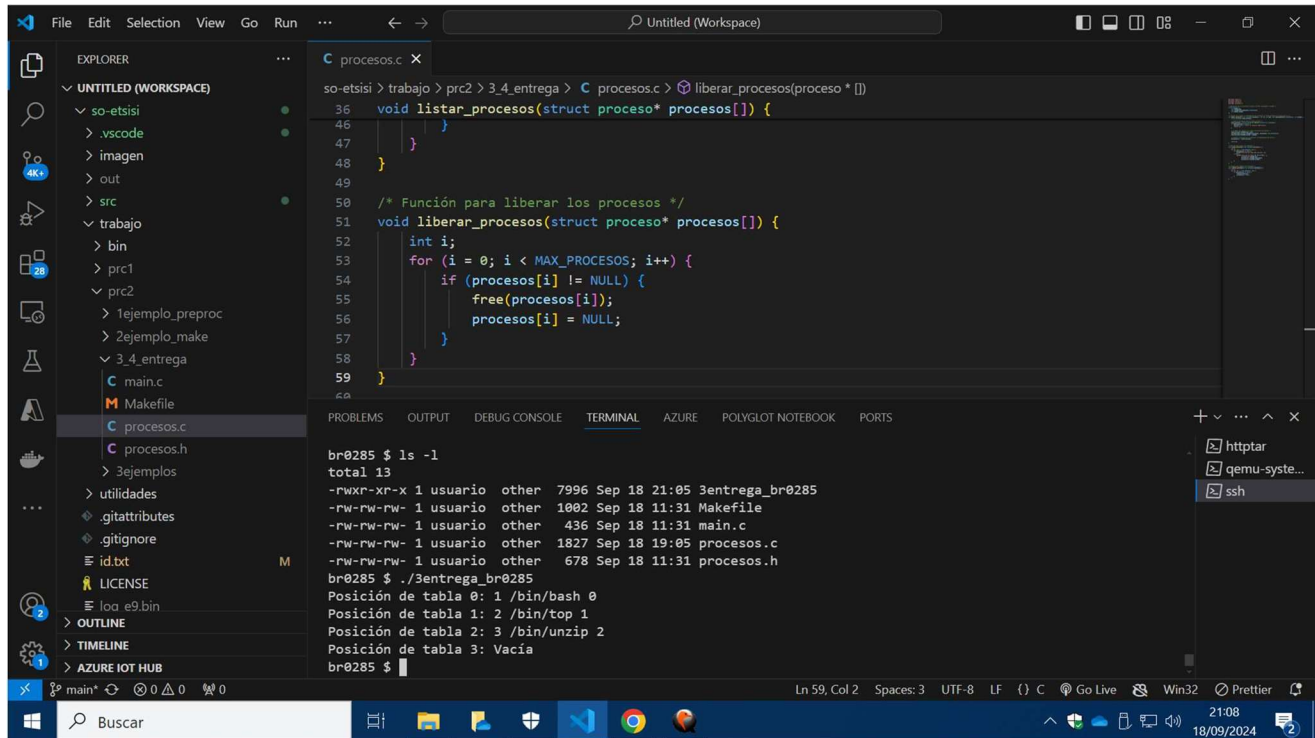
Terminal (Bottom):

```
Yzan Martinezz Manzano - br0285
br0285 $
```

Taskbar (Bottom):

- Windows Start button
- Search bar: `Buscar`
- Taskbar buttons: `trabajo`, `procesos.c - so-etsi...`, `QEMU (br0285)`
- System tray: `11:20`, `20/09/2024`, `3` notifications

A entregar – 4. Funciones C para gestión de array Realiza una captura de pantalla en la que deberá aparecer el código del fichero procesos.c y la ejecución correcta en la terminal con ssh.



The screenshot displays the Visual Studio Code interface with a workspace named 'Untitled (Workspace)'. The Explorer panel on the left shows a project structure with folders like 'so-etsisi', 'vscod', 'imagen', 'out', 'src', 'trabajo', 'bin', 'prc1', 'prc2', '1ejemplo_preproc', '2ejemplo_make', '3_4_entrega', and files like 'main.c', 'Makefile', 'procesos.c', 'procesos.h', '3ejemplos', 'utilidades', '.gitattributes', '.gitignore', 'id.txt', 'LICENSE', 'loc_e9.bin', 'OUTLINE', 'TIMELINE', and 'AZURE IOT HUB'. The 'procesos.c' file is open in the editor, showing the following C code:

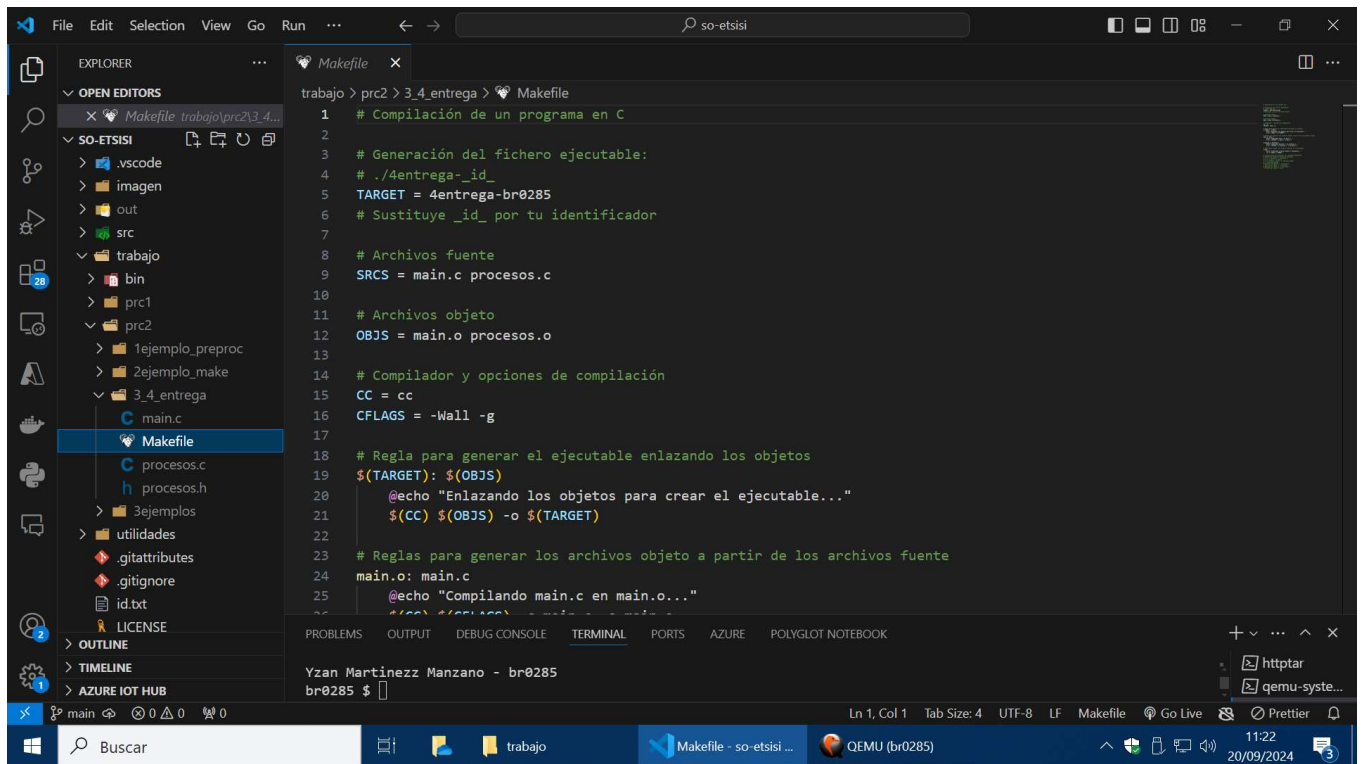
```
so-etsisi > trabajo > prc2 > 3_4_entrega > C procesos.c > liberar_procesos(proceso * [])  
36 void listar_procesos(struct proceso* procesos[]) {  
46 }  
47 }  
48 }  
49 }  
50 /* Función para liberar los procesos */  
51 void liberar_procesos(struct proceso* procesos[]) {  
52     int i;  
53     for (i = 0; i < MAX_PROCESOS; i++) {  
54         if (procesos[i] != NULL) {  
55             free(procesos[i]);  
56             procesos[i] = NULL;  
57         }  
58     }  
59 }
```

The TERMINAL panel at the bottom shows the execution of the program. The user runs 'ls -l' and '3entrega_br0285', resulting in the following output:

```
br0285 $ ls -l  
total 13  
-rwxr-xr-x 1 usuario other 7996 Sep 18 21:05 3entrega_br0285  
-rw-rw-rw- 1 usuario other 1002 Sep 18 11:31 Makefile  
-rw-rw-rw- 1 usuario other 436 Sep 18 11:31 main.c  
-rw-rw-rw- 1 usuario other 1827 Sep 18 19:05 procesos.c  
-rw-rw-rw- 1 usuario other 678 Sep 18 11:31 procesos.h  
br0285 $ ./3entrega_br0285  
Posición de tabla 0: 1 /bin/bash 0  
Posición de tabla 1: 2 /bin/top 1  
Posición de tabla 2: 3 /bin/unzip 2  
Posición de tabla 3: Vacía  
br0285 $
```

The status bar at the bottom indicates the current file is 'main.c' at line 59, column 2, with 3 spaces, UTF-8 encoding, LF line endings, and C language mode. The system tray shows the date and time as 21:08 on 18/09/2024.

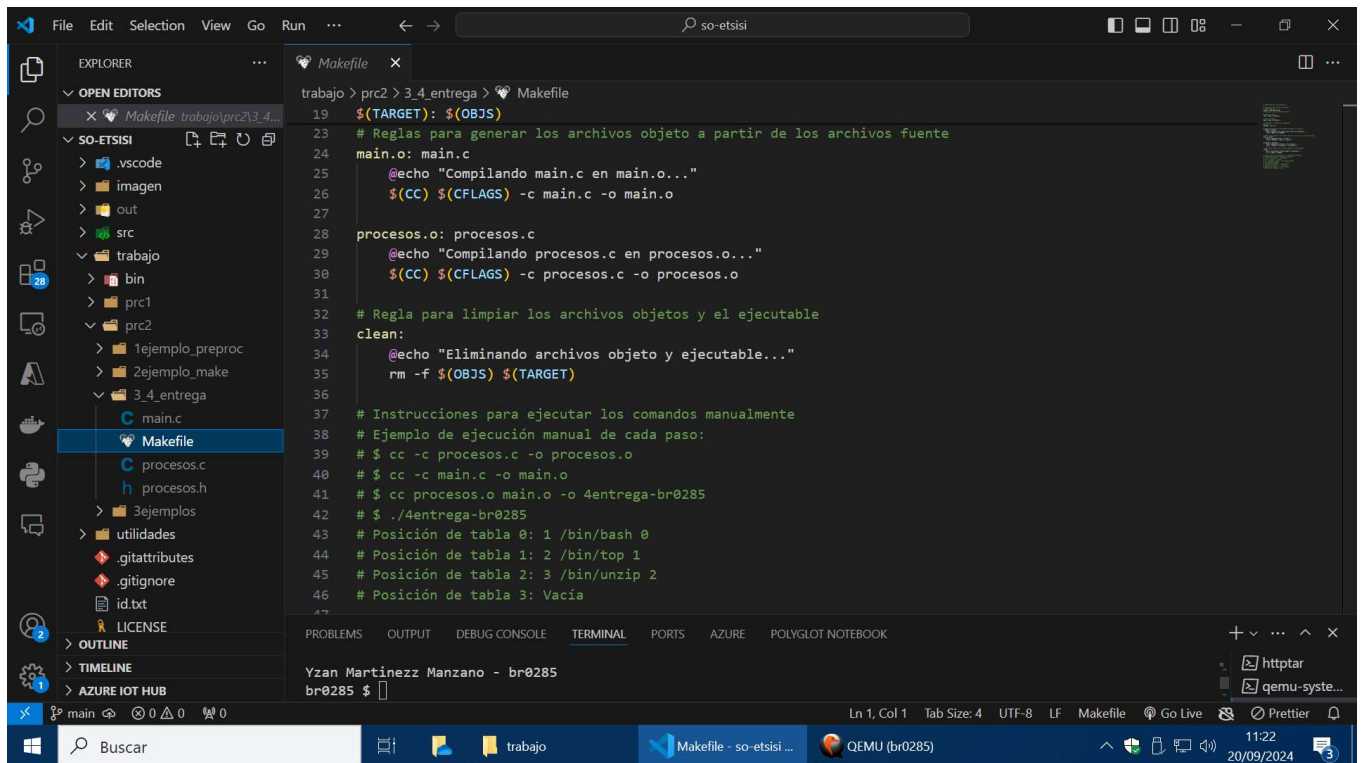
A entregar – 5. Makefile Realiza una captura de pantalla en la que se vea el contenido del fichero Makefile y en la terminal la ejecución de make y la salida del fichero generado. Asegúrate de hacer antes un make clean.



This screenshot shows the Visual Studio Code editor with the 'Makefile' file open. The file is located at 'trabajo > prc2 > 3.4_entrega > Makefile'. The Makefile content is as follows:

```
1 # Compilación de un programa en C
2
3 # Generación del fichero ejecutable:
4 # ./4entrega_id_
5 TARGET = 4entrega-br0285
6 # Sustituye _id_ por tu identificador
7
8 # Archivos fuente
9 SRCS = main.c procesos.c
10
11 # Archivos objeto
12 OBJS = main.o procesos.o
13
14 # Compilador y opciones de compilación
15 CC = cc
16 CFLAGS = -Wall -g
17
18 # Regla para generar el ejecutable enlazando los objetos
19 $(TARGET): $(OBJS)
20     @echo "Enlazando los objetos para crear el ejecutable..."
21     $(CC) $(OBJS) -o $(TARGET)
22
23 # Reglas para generar los archivos objeto a partir de los archivos fuente
24 main.o: main.c
25     @echo "Compilando main.c en main.o..."
26     $(CC) $(CFLAGS) -c main.c -o main.o
27
28 procesos.o: procesos.c
29     @echo "Compilando procesos.c en procesos.o..."
30     $(CC) $(CFLAGS) -c procesos.c -o procesos.o
31
32 # Regla para limpiar los archivos objetos y el ejecutable
33 clean:
34     @echo "Eliminando archivos objeto y ejecutable..."
35     rm -f $(OBJS) $(TARGET)
36
37 # Instrucciones para ejecutar los comandos manualmente
38 # Ejemplo de ejecución manual de cada paso:
39 # $ cc -c procesos.c -o procesos.o
40 # $ cc -c main.c -o main.o
41 # $ cc procesos.o main.o -o 4entrega-br0285
42 # $ ./4entrega-br0285
43 # Posición de tabla 0: 1 /bin/bash 0
44 # Posición de tabla 1: 2 /bin/top 1
45 # Posición de tabla 2: 3 /bin/unzip 2
46 # Posición de tabla 3: Vacía
```

The terminal at the bottom shows the command 'trabajo > prc2 > 3.4_entrega > Makefile' and the output 'Yzan Martinezz Manzano - br0285'.



This screenshot shows the Visual Studio Code editor with the 'Makefile' file open. The file is located at 'trabajo > prc2 > 3.4_entrega > Makefile'. The Makefile content is as follows:

```
19 $(TARGET): $(OBJS)
20     @echo "Enlazando los objetos para crear el ejecutable..."
21     $(CC) $(OBJS) -o $(TARGET)
22
23 # Reglas para generar los archivos objeto a partir de los archivos fuente
24 main.o: main.c
25     @echo "Compilando main.c en main.o..."
26     $(CC) $(CFLAGS) -c main.c -o main.o
27
28 procesos.o: procesos.c
29     @echo "Compilando procesos.c en procesos.o..."
30     $(CC) $(CFLAGS) -c procesos.c -o procesos.o
31
32 # Regla para limpiar los archivos objetos y el ejecutable
33 clean:
34     @echo "Eliminando archivos objeto y ejecutable..."
35     rm -f $(OBJS) $(TARGET)
36
37 # Instrucciones para ejecutar los comandos manualmente
38 # Ejemplo de ejecución manual de cada paso:
39 # $ cc -c procesos.c -o procesos.o
40 # $ cc -c main.c -o main.o
41 # $ cc procesos.o main.o -o 4entrega-br0285
42 # $ ./4entrega-br0285
43 # Posición de tabla 0: 1 /bin/bash 0
44 # Posición de tabla 1: 2 /bin/top 1
45 # Posición de tabla 2: 3 /bin/unzip 2
46 # Posición de tabla 3: Vacía
```

The terminal at the bottom shows the command 'trabajo > prc2 > 3.4_entrega > Makefile' and the output 'Yzan Martinezz Manzano - br0285'.

